

Perceptual Image Hashing Using CNN

A Project Report Submitted in
Partial Fulfillment of the Requirements for the Final
Semester B.Tech. Project

by

Mst. Habiba Akter Bithi	2012162
Parijat Biswas Pritom	2012164
Muhamad Shakhawat Hossain	2012172

Under the Supervision of
Mr. Prabhakar Sarma Neog



Department of Computer Science & Engineering
NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR
May, 2024



Department of Computer Science & Engineering

NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR

Certificate

It is certified that the work contained in this thesis entitled “**Perceptual Image Hashing Using CNN**” submitted by **Mst. Habiba Akter Bithi (2012162)**, **Parijat Biswas Pritom (2012164)**, and **Muhammad Shakhawat Hossain (2012172)** for the B.Tech. End Semester Project Examination, May 2024, is based on their work carried out under my supervision.

Place: NIT Silchar,

Date: 15/05/2024

Mr. Prabhakar Sarma Neog
Computer Science & Engineering
National Institute of Technology Silchar



Department of Computer Science & Engineering

NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR

Declaration

Thesis Title: Perceptual Image Hashing Using CNN

Degree for which the thesis is submitted: Bachelor of Technology

We declare that the presented thesis represents largely our own ideas and work in our own words. Where others' ideas or words have been included, we have adequately cited them and listed them in the reference materials. The thesis has been prepared without resorting to plagiarism. We have adhered to all principles of academic honesty and integrity. No falsified or fabricated data have been presented in the thesis. We understand that any violation of the above will cause disciplinary action by the Institute, including revoking the conferred degree, if conferred, and can also evoke penal action from the sources that have not been properly cited or from whom proper permission has not been taken.

Date: 15/05/2024

Mst. Habiba Akter Bithi
Parijat Biswas Pritom
Muhamad Shakhawat Hossain

Acknowledgements

We take this opportunity to express our sincere gratitude to our supervisor, Mr. Prabhakar Sarma Neog, Department of Computer Science and Engineering, National Institute of Technology Silchar, for his continuous inspiration and valuable guidance at every stage of this work. His expertise, dedication, and mentorship played a pivotal role in the progress of our project, and we deeply appreciate his patience, constructive feedback, and encouragement, which enriched the project and contributed significantly to our academic and professional growth.

We would also like to thank all the faculty members of the Department of Computer Science and Engineering, National Institute of Technology Silchar, for their administrative support during the various phases of this work.

Mst. Habiba Akter Bithi
Parijat Biswas Pritom
Muhamad Shakhawat Hossain

Abstract

Perceptual image hashing generates compact binary representations of digital images that remain similar under content-preserving distortions such as compression, noise, filtering and geometric changes, while differing for images of different content. This makes such hashes useful for image authentication, copy detection and image retrieval. This project presents a perceptual image hashing scheme built on a convolutional neural network (CNN). An input image is resized to 224×224 pixels and passed through the convolutional base of a VGG16 network pretrained on ImageNet; global average pooling of the final feature map yields a 512-dimensional feature vector, which is binarised around its mean to produce a 512-bit hash. Similarity between two images is measured by the normalised Hamming distance between their hashes and compared with a decision threshold.

The scheme was evaluated on 50 images. Each image was subjected to ten families of content-preserving transformations (96 parameter settings in total), giving 4,800 transformed images and, by construction, 4,800 similar and 235,200 dissimilar image pairs. Sweeping the decision threshold over 200 values gives an area under the ROC curve (AUC) of 0.991 and an equal error rate (EER) of approximately 4.3%. At the threshold selected by Youden's J statistic ($\epsilon^* = 0.2613$), the true positive rate is 96.3% and the false positive rate is 4.7%. Detection was most reliable for gamma correction, colour changes and rotation up to 15° , and least reliable for speckle noise and Gaussian and median filtering. The results indicate that features from a frozen pretrained VGG16, combined with simple mean-based binarisation, provide a useful trade-off between robustness and discrimination on this dataset, while the low precision at high recall highlights the need for larger-scale evaluation and task-specific training.

Keywords: perceptual image hashing; convolutional neural network; VGG16; Hamming distance; ROC curve; content-preserving attacks.

Table of Contents

1 Introduction	1
1.1 Background	1
1.2 Perceptual Image Hashing	1
1.3 Motivation	1
1.4 Problem Statement	2
1.5 Research Objectives	2
1.6 Scope and Limitations	3
1.7 Contributions	3
1.8 Organisation of the Thesis	3
2 Literature Review	4
2.1 Traditional Image Hashing	4
2.2 Perceptual Hashing with Hand-Crafted Features	4
2.3 CNN-Based Image Hashing	4
2.4 Deep Networks as Feature Extractors	4
2.5 Research Gap and Motivation for the Proposed Approach	5
3 Proposed Methodology	6
3.1 Overall System Architecture	6
3.2 Dataset	7
3.3 Image Pre-processing	7
3.4 CNN Architecture	8
3.4.1 VGG16 Convolutional Base	8
3.4.2 Other Architectures	8
3.5 Feature Extraction with Global Average Pooling	8
3.6 Hash Generation and Binarisation	8
3.7 Similarity Measurement	9
3.8 Image Transformations	9
3.9 Ground-Truth Construction	11
3.10 Evaluation Pipeline and Metrics	12
3.11 Implementation Environment	13
4 Experimental Results and Discussion	14
4.1 Experimental Setup	14
4.2 Hashing Results	14
4.3 Hamming Distance Analysis	15
4.4 Transformation-Wise Results	15
4.5 Threshold Analysis	17
4.6 ROC Curve, AUC and EER	18
4.7 Confusion Matrix	19
4.8 Discussion	20
5 Conclusion and Future Work	22
5.1 Conclusion	22
5.2 Future Work	22
References	23

List of Figures

Figure 3.1: Hash generation pipeline: pre-processing, VGG16 feature extraction with global average pooling, and mean-threshold binarisation	6
Figure 3.2: Sample source images from the dataset: (a) cow, (b) tiger and (c) flower	7
Figure 3.3: Dataset construction and evaluation workflow	12
Figure 4.1: Distribution of the normalised Hamming distance for similar and dissimilar pairs	15
Figure 4.2: Mean normalised Hamming distance of similar pairs for each transformation family	16
Figure 4.3: TPR, FPR, precision and F1 as functions of the decision threshold ϵ	18
Figure 4.4: ROC curve of the VGG16-based perceptual hashing scheme (AUC = 0.991)	19
Figure 4.5: Confusion matrix at the Youden-optimal threshold $\epsilon^* = 0.2613$ (rows: actual class; columns: predicted class)	20

List of Tables

Table 3.1: Configuration of the hash generation pipeline as implemented	7
Table 3.2: Image transformations, parameter values and implementation details	10
Table 3.3: Number of transformation settings per image and resulting image pairs	11
Table 4.1: Summary of the experimental setup	14
Table 4.2: Normalised Hamming distance statistics for similar and dissimilar pairs	15
Table 4.3: Similar-pair distance statistics per transformation, and detection rate at $\epsilon^* = 0.2613$	16
Table 4.4: Performance at selected operating points	17
Table 4.5: Performance over a regular grid of thresholds (nearest swept value)	17
Table 4.6: Summary of the evaluation measures	19
Table 4.7: Confusion matrix at $\epsilon^* = 0.2613$	20

List of Algorithms

Algorithm 3.1: Hash generation.	9
Algorithm 3.2: Dataset construction and ground-truth pairs.	11
Algorithm 3.3: Threshold sweep and evaluation.	13

List of Abbreviations

AUC	Area Under the (ROC) Curve
CNN	Convolutional Neural Network
EER	Equal Error Rate
FN	False Negative
FNR	False Negative Rate
FP	False Positive
FPR	False Positive Rate
PIH	Perceptual Image Hashing
ReLU	Rectified Linear Unit
ROC	Receiver Operating Characteristic
TN	True Negative
TP	True Positive
TPR	True Positive Rate
VGG	Visual Geometry Group (network family)

1 Introduction

1.1 Background

Digital images are easily copied, edited, recompressed, and redistributed. Cryptographic hash functions such as SHA-256 can verify that two files are bit-for-bit identical, but they are unsuitable for judging whether two images *look* the same: a single changed pixel, or a harmless re-encoding, produces a completely different digest. Perceptual image hashing addresses this limitation by mapping an image to a short binary code that depends on its visual content rather than on its exact bytes. Perceptual hashing for image authentication has been surveyed in [1], and typical applications include image authentication, copy detection, and content-based image retrieval.

1.2 Perceptual Image Hashing

A perceptual image hashing scheme has two competing requirements. *Robustness* means that an image and a content-preserving variant of it (for example, a slightly rotated, blurred, noisy, or recompressed copy) should have hashes that are close under a chosen distance. *Discrimination* means that images with different content should have hashes that are far apart. A scheme is useful only if a single decision threshold on the hash distance can satisfy both requirements simultaneously; the performance trade-off between the two is usually summarized with an ROC curve.

Most schemes follow a common pipeline: preprocessing to a canonical size and format, feature extraction, quantization or binarization of the features into a fixed-length code, and comparison of codes with the Hamming distance. Earlier schemes extract hand-crafted statistical or texture features [4], [5]. In recent years CNNs have been used as feature extractors because they learn hierarchical visual features directly from data [3].

1.3 Motivation

The motivation for a CNN-based approach comes from limitations that are commonly attributed to hand-crafted schemes and from properties of deep networks:

- **Robustness to transformations.** Hand-crafted features must be designed to survive each class of distortion. Deep features learned from large image collections may respond to image content in a way that is less sensitive to such distortions.
- **Discriminative capability.** Distinguishing perceptually different images while tolerating benign modifications is central to authentication, retrieval and copyright

protection. Deep networks can represent complex patterns and may therefore separate different content more effectively.

- **Automated feature extraction.** A pretrained CNN removes the need for manual feature engineering and domain-specific design.
- **Potential for deep hashing.** Deep hashing methods integrate hash-code generation into the network so that feature extraction and quantization are optimized jointly [2], [3]. The present work uses the simpler setting of a frozen pretrained network and thereby provides a baseline against which such trained schemes can be compared.

1.4 Problem Statement

Perceptual image hashing continues to face several open problems:

- (i) selecting transformations, invariant features, decompositions or statistical properties of images that are resilient against most possible attacks;
- (ii) for CNN-based schemes, selecting constrained loss functions that supplement the network loss so that hash distances between similar images are penalised and distances between dissimilar images are rewarded;
- (iii) achieving robustness against individual attacks and combinations of attacks; and
- (iv) balancing the trade-off between the robustness and the discrimination capability of the hash function.

This project addresses the first and last of these problems in a restricted setting: it studies how well the features of a frozen, ImageNet-pretrained VGG16, binarised with a simple mean threshold, separate transformed copies of an image from different images. Training with task-specific loss functions (item 2) is not part of the implemented system and is discussed as future work in Chapter 5.

1.5 Research Objectives

The objectives of this work are:

1. to survey existing literature on perceptual image hashing and CNN-based hashing;
2. to design a perceptual hashing scheme that maps an image to a fixed-length binary code using a pretrained CNN;
3. to implement the scheme as a complete, reproducible pipeline covering feature extraction, binarisation and Hamming-distance comparison;
4. to construct an evaluation dataset by applying ten families of content-preserving transformations to a set of source images and building labelled similar and dissimilar pairs; and

5. to evaluate the trade-off between robustness and discrimination through a threshold sweep, ROC curve, AUC, EER and confusion-matrix analysis.

1.6 Scope and Limitations

The implemented scheme uses the VGG16 convolutional base with ImageNet weights and does not perform any training or fine-tuning. The evaluation uses 50 source images and the ten transformation families listed in Table 3.2. Other attacks (for example cropping, scaling or combinations of transformations) are not evaluated, and applications of the hash to a downstream task such as copy detection are not tested in this work. The code supports VGG19 as an alternative backbone, but no VGG19 results are reported.

1.7 Contributions

- An end-to-end implementation of a VGG16-based perceptual hashing pipeline that produces 512-bit hashes using global average pooling and mean-threshold binarisation.
- A reproducible evaluation protocol that generates 4,800 transformed images from 50 sources, constructs 240,000 labelled image pairs and evaluates them by threshold sweeping.
- An empirical characterisation of the scheme, reporting AUC, EER, threshold-dependent TPR, FPR, precision and F1, and a per-transformation analysis of robustness.

1.8 Organisation of the Thesis

Chapter 2 reviews related work. Chapter 3 describes the proposed methodology, including the hashing pipeline, the dataset, the transformations, the construction of ground truth and the evaluation metrics. Chapter 4 reports and discusses the experimental results. Chapter 5 concludes the thesis and outlines future work.

2 Literature Review

This chapter situates the present work among existing approaches to perceptual image hashing. Only the works listed in the reference list are discussed, and they are described at the level supported by their titles and publication venues.

2.1 Traditional Image Hashing

Image hashing was introduced to provide compact, content-based identifiers for images. Unlike cryptographic hashes, which are intended to change completely when the input changes, image hashes are designed to tolerate content-preserving processing while reacting to real changes in content. A survey of perceptual hashing for image authentication is given in [1].

2.2 Perceptual Hashing with Hand-Crafted Features

Conventional schemes derive the hash from hand-designed image descriptors. Srivastava *et al.* [4] propose a robust image hashing scheme based on statistical features for copy detection. Huang and Liu [5] present a hashing method oriented towards robustness and discrimination that combines texture information with an invariant vector distance. Both approaches rely on features chosen by the designer rather than learned from data.

2.3 CNN-Based Image Hashing

CNNs have been used to learn the features from which hashes are produced. Qin *et al.* [3] propose a perceptual image hashing scheme for content authentication that uses a convolutional neural network trained with multiple constraints. Yuan *et al.* [2] study central similarity quantization for efficient image and video retrieval, an example of learning binary codes for retrieval with deep networks. In both cases the network is trained for the hashing objective, in contrast to the frozen pretrained backbone used here.

2.4 Deep Networks as Feature Extractors

VGG16 [6] is a deep CNN with a homogeneous stack of small convolutional filters. Trained on the ImageNet dataset [7], its convolutional base produces features that are widely reused for other visual tasks. Removing the classification head and applying global average pooling to the last convolutional feature map yields a fixed-length descriptor whose size (512 for VGG16) is independent of the input image size.

2.5 Research Gap and Motivation for the Proposed Approach

The reviewed hand-crafted methods require the designer to anticipate the distortions that the hash should survive, while the reviewed trained CNN methods require a training procedure with carefully constructed constraints or losses. It is therefore of practical interest to establish how far a pretrained network can be used *without* any hashing-specific training, using only a fixed feature extractor and a simple binarisation rule. This is the question studied in this thesis. The answer provides a baseline for later, trained schemes, and it is evaluated with a large number of labelled pairs and standard ROC-based metrics.

Note that no direct experimental comparison with the methods in [1]–[5] is carried out in this thesis; the comparison is therefore qualitative only.

3 Proposed Methodology

3.1 Overall System Architecture

The proposed scheme consists of three stages: (i) pre-processing, in which an image is resized and normalised for the network; (ii) feature extraction, in which the VGG16 convolutional base and global average pooling produce a 512-dimensional feature vector; and (iii) hash generation, in which the feature vector is binarised into a 512-bit hash. Two images are compared by the normalised Hamming distance of their hashes, and they are declared similar when the distance is below a threshold ϵ . Figure 3.1 shows the hash generation pipeline, and Table 3.1 summarises its configuration as implemented in the accompanying code.

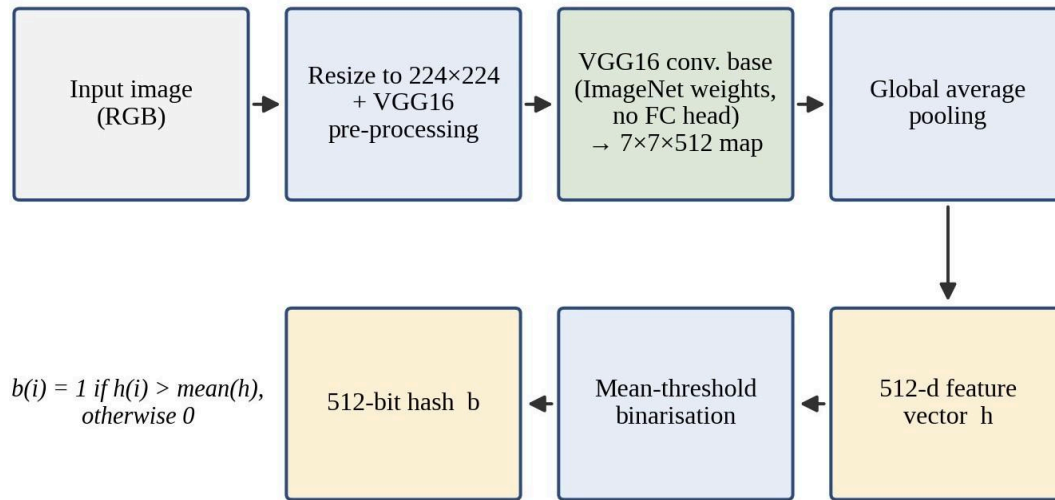


Figure 3.1: Hash generation pipeline: pre-processing, VGG16 feature extraction with global average pooling, and mean-threshold binarisation.

Table 3.1: Configuration of the hash generation pipeline as implemented.

Stage	Setting	Source file
Input size	$224 \times 224 \times 3$ (RGB)	feature_extractor.py
Pre-processing	VGG16 preprocess_input (Keras)	feature_extractor.py
Backbone	VGG16, ImageNet weights, classification head removed	feature_extractor.py
Pooling	Global average pooling of the $7 \times 7 \times 512$ feature map	feature_extractor.py
Feature vector	512-dimensional, real-valued	feature_extractor.py
Binarisation	Mean of the vector used as threshold	hash_utils.py
Hash length	512 bits	hash_utils.py
Similarity	Normalised Hamming distance	hash_utils.py

3.2 Dataset

The experimental dataset consists of 50 source images (named 01.jpg to 50.jpg in the results files). According to the project repository, they span animals, plants and everyday scenes; three examples are shown in Figure 3.2. The source of the images and their original resolutions are [Information not available in provided materials].



Figure 3.2: Sample source images from the dataset: (a) cow, (b) tiger, and (c) flower.

3.3 Image Pre-processing

Each image is loaded from disk, resized to 224×224 pixels and converted to an array with three colour channels. The network-specific pre-processing function of VGG16 (mean subtraction and channel reordering, as provided by Keras) is then applied. Because all images are resized to the same fixed size, every image yields a feature vector of the same length regardless of its original dimensions.

3.4 CNN Architecture

3.4.1 VGG16 Convolutional Base

VGG16 [6] was designed for image classification. It stacks convolutional layers with small 3×3 filters and ReLU activations in five convolutional blocks, each followed by max-pooling, and ends with fully connected classification layers. In this work only the convolutional base is used: the fully connected head is removed, and the network is initialised with the weights obtained by ImageNet training [7]. No training or fine-tuning is performed, so the mapping from image to feature vector is fixed and deterministic. For a $224 \times 224 \times 3$ input, the output of the final convolutional block is a $7 \times 7 \times 512$ feature map.

3.4.2 Other Architectures

The feature extractor also accepts VGG19 as a backbone through the same interface, again with average pooling and a 512-dimensional output. Only VGG16 was used to produce the results in Chapter 4; therefore, no comparison between backbones is reported. AlexNet is mentioned in the source-code documentation but is not implemented, and it is not used in this work.

3.5 Feature Extraction with Global Average Pooling

Let I denote an input image, x the pre-processed $224 \times 224 \times 3$ array, and Φ the VGG16 convolutional base. The final feature map is

$$F = \Phi(x) \in \mathbb{R}^{7 \times 7 \times 512} \quad (3.1)$$

Global average pooling reduces each of the 512 channels to a single number by averaging over the 7×7 spatial positions, giving the feature vector $h = (h_1, \dots, h_{512})$:

$$h_k = (1/49) \sum_{i=1}^7 \sum_{j=1}^7 F_{i,j,k}, \quad k = 1, \dots, 512 \quad (3.2)$$

3.6 Hash Generation and Binarization

The real-valued feature vector is converted into a binary hash using an image-adaptive threshold equal to the mean of the vector:

$$\mu = (1/n) \sum_{k=1}^n h_k, \quad n = 512 \quad (3.3)$$

$$b_k = 1 \text{ if } h_k > \mu, \quad b_k = 0 \text{ if } h_k \leq \mu \quad (3.4)$$

The resulting hash is $b = (b_1, \dots, b_{512}) \in \{0,1\}^{512}$. Because the threshold is computed from each image's own feature vector, it adapts to overall changes in feature magnitude, such as those caused by brightness changes. Algorithm 3.1 summarizes the procedure.

Algorithm 3.1: Hash generation.

Input: image file paths for all original and transformed images; backbone (default VGG16).

Output: hash database of (image name, 512-bit hash).

- Load the pretrained convolutional base with global average pooling.
- **for** each image I **do**
 - Resize I to 224×224 and apply VGG16 pre-processing to obtain x .
 - Compute the 512-dimensional feature vector h (Equations 3.1–3.2).
 - Compute $\mu = \text{mean}(h)$ (Equation 3.3).
 - Set $b_k = 1$ if $h_k > \mu$, otherwise 0 (Equation 3.4).
 - Store (name of I , b) in the database.
- **end for**
- Save the database to a CSV file.

3.7 Similarity Measurement

The Hamming distance between two hashes b^A and b^B of length n is the number of bit positions at which they differ. It is normalised by the hash length so that it lies in $[0, 1]$:

$$d_H(b^A, b^B) = \sum_{k=1}^n 1[b_k^A \neq b_k^B] \quad (3.5)$$

$$d(A, B) = d_H(b^A, b^B) / n \quad (3.6)$$

Two images are declared similar if their normalised distance is smaller than a threshold ε , and dissimilar otherwise:

$$\hat{y}(A, B; \varepsilon) = 1 \text{ if } d(A, B) < \varepsilon, \text{ 0 otherwise} \quad (3.7)$$

3.8 Image Transformations

To emulate content-preserving processing, ten families of transformations are applied to every source image, each at several parameter values (Table 3.2). The parameter grids are defined in the file `transformations.py` and correspond to the parameter grids of the original project report. Transformed images are saved in the same file format as the source image, so that each of them also undergoes the re-encoding performed by the image-saving routine.

Table 3.2: Image transformations, parameter values and implementation details.

Transformation	Parameter values	Implementation	Library
Rotation	Angle (°): 1, 2, 3, 4, 6, 8, 10, 12, 13, 15	Bicubic rotation, canvas size kept, grey fill (128, 128, 128)	Pillow
Speckle noise	Variance: 0.01, 0.02, 0.03, 0.04, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3	Multiplicative speckle noise, output clipped	scikit-image
Gaussian filter (variance)	Variance: 0.01, 0.02, 0.03, 0.04, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3	Gaussian blur per channel with $\sigma = 10 \cdot \sqrt{\text{variance}}$	SciPy
Gaussian filter (size)	Filter size: 1, 3, 5, 7, 9, 11, 13, 15, 17, 19	Gaussian blur with radius = $\max(\text{size} / 3, 0.1)$	Pillow
Median filter	Filter size: 1, 3, 5, 7, 9, 11, 13, 15, 17, 19	Square median window, applied per channel	SciPy
Circular median filter	Ball radius: 1, 1.5, 2, 2.2, 2.5, 2.6, 2.7, 3, 3.5	Disk-shaped median footprint of radius $\text{round}(r)$, minimum 1	SciPy
Gamma correction	Gamma: 0.55, 0.65, 0.75, 0.85, 0.95, 1.05, 1.15, 1.25, 1.35, 1.45	Power law: $255 \cdot (\text{in}/255)^{\gamma}$	NumPy
Brightness	Factor: 0.7, 0.8, 1, 1.2, 1.5, 2, 2.2, 2.5	Brightness enhancement	Pillow
Colour	Factor: 0.2, 0.5, 1, 1.2, 1.3, 1.5, 2, 2.3, 2.5, 3	Colour (saturation) enhancement	Pillow
Sharpness	Factor: 0.1, 0.5, 1, 2, 3, 5, 7, 8, 10	Sharpness enhancement	Pillow

Two properties of the implementation should be kept in mind when interpreting the results. First, the circular median filter rounds the ball radius to an integer (Python rounding), so that the settings 1.5, 2, 2.2 and 2.5 all use radius 2 and the settings 2.6, 2.7 and 3 all use radius 3; the nine parameter values therefore correspond to four distinct filter footprints (radii 1, 2, 3 and 4). This is confirmed by the results: the mean distances of the settings within each group are identical. Second, some settings are nominally neutral (factor 1 for brightness, colour and sharpness; window size 1 for the median filter); the corresponding transformed images differ from their originals only through the re-encoding described above.

3.9 Ground-Truth Construction

Ground truth is defined by construction. For each source image A , a *similar* pair (label 1) is formed by A and each of its own transformed variants. A *dissimilar* pair (label 0) is formed by A and each transformed variant that was derived from a different source image. The pair list is stored as (image_a, image_b, label); the Hamming distance is computed later during evaluation. Since the transformation grid contains 96 settings (Table 3.3), every source image has 96 similar pairs and $49 \times 96 = 4,704$ dissimilar pairs, so the ratio of similar to dissimilar pairs is 1 : 49. Algorithm 3.2 summarises the construction.

Table 3.3: Number of transformation settings per image and resulting image pairs.

Transformation	Settings per image	Similar pairs	Dissimilar pairs
Rotation	10	500	24,500
Speckle noise	10	500	24,500
Gaussian filter (variance)	10	500	24,500
Gaussian filter (size)	10	500	24,500
Median filter	10	500	24,500
Circular median filter	9	450	22,050
Gamma correction	10	500	24,500
Brightness	8	400	19,600
Colour	10	500	24,500
Sharpness	9	450	22,050
Total	96	4,800	235,200

Algorithm 3.2: Dataset construction and ground-truth pairs.

Input: source images; transformation table (Table 3.2).
Output: transformed images; transformation log; pair list (image_a, image_b, label).

- **for** each source image A **do**
 - **for** each transformation T and each parameter value p of T **do**
 - Save $T(A, p)$ as a transformed image and record $(A, T(A, p), T, p)$ in the log.
 - **end for**
- **end for**
- **for** each source image A **do**
 - Add $(A, V, 1)$ for every transformed variant V of A .
 - Add $(A, V, 0)$ for every transformed variant V of every other source image.
- **end for**

3.10 Evaluation Pipeline and Metrics

Figure 3.3 gives an overview of the evaluation workflow. After the hash database has been computed, the normalised Hamming distance is attached to every labelled pair (Equation 3.6). The decision threshold ϵ is then swept over 200 equally spaced values in $[0, 1]$, $\epsilon_m = m / 199$ for $m = 0, \dots, 199$. At each threshold, Equation 3.7 gives a prediction for every pair, and the entries of the confusion matrix are counted: true positives (TP; similar pairs predicted similar), false positives (FP; dissimilar pairs predicted similar), false negatives (FN; similar pairs predicted dissimilar) and true negatives (TN; dissimilar pairs predicted dissimilar).

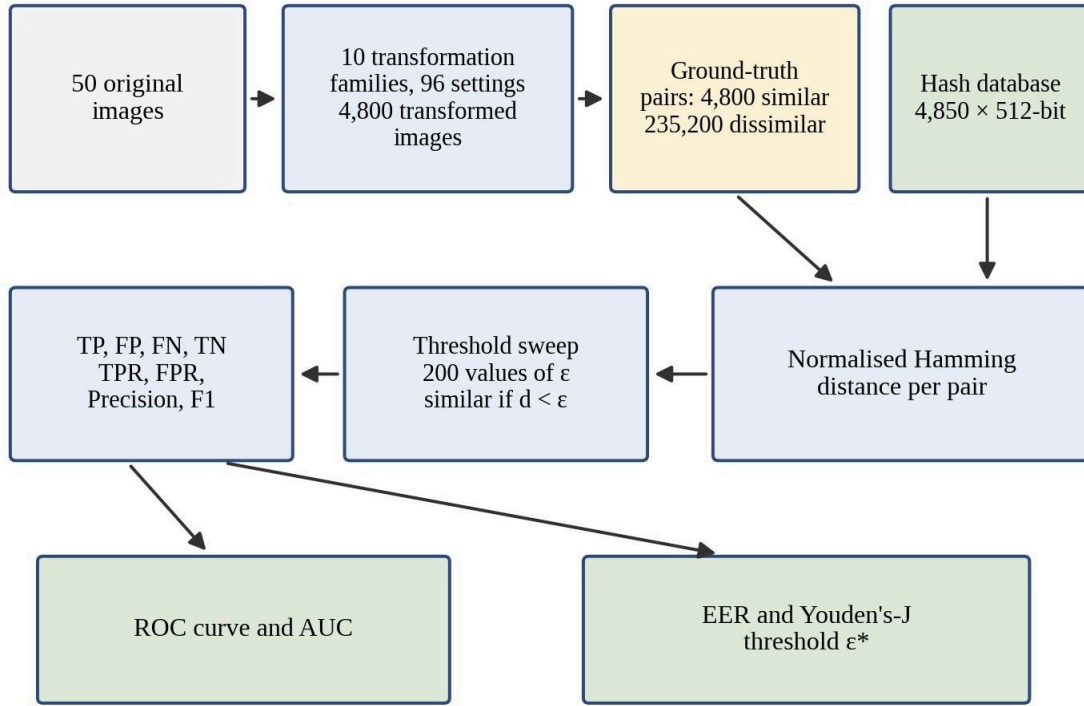


Figure 3.3: Dataset construction and evaluation workflow.

From these counts the following measures are computed at each threshold:

$$\text{TPR} = \text{TP} / (\text{TP} + \text{FN}) \quad (3.8)$$

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN}) \quad (3.9)$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \quad (3.10)$$

$$\text{F1} = 2 \cdot \text{Precision} \cdot \text{TPR} / (\text{Precision} + \text{TPR}) \quad (3.11)$$

The ROC curve plots TPR against FPR over all thresholds. The area under the curve is computed by the trapezoidal rule over the points sorted by FPR:

$$AUC = \sum_j \frac{1}{2} (TPR_j + TPR_{j+1}) (FPR_{j+1} - FPR_j) \quad (3.12)$$

The equal error rate (EER) is the value of the FPR at the threshold where the false positive rate and the false negative rate $FNR = 1 - TPR$ are closest to each other. Youden's J statistic and the corresponding optimal threshold are

$$J(\epsilon) = TPR(\epsilon) - FPR(\epsilon), \epsilon^* = \arg \max_{\epsilon} J(\epsilon) \quad (3.13)$$

The procedure is summarised in Algorithm 3.3.

Algorithm 3.3: Threshold sweep and evaluation.

Input: pair list with labels; hash database.
Output: threshold-metrics table; ROC curve; AUC; EER; ϵ^* .

- For every pair, look up both hashes and compute d (Equation 3.6).
- **for** $m = 0, \dots, 199$ **do**
 - $\epsilon \leftarrow m / 199$; predict similar for every pair with $d < \epsilon$.
 - Count TP, FP, FN, TN; compute TPR, FPR, Precision and F1.
- **end for**
- Plot TPR against FPR; compute AUC by the trapezoidal rule.
- Compute the EER and $\epsilon^* = \arg \max (TPR - FPR)$.

3.11 Implementation Environment

The pipeline is implemented in Python. Feature extraction uses TensorFlow/Keras; image transformations use Pillow, scikit-image and SciPy; and the evaluation uses NumPy, pandas and Matplotlib. The stages are implemented as separate scripts (dataset generation, hash computation and evaluation) and can be run end to end from a single entry point; the source code is available at <https://github.com/Habibabithi/perceptual-image-hashing-cnn>. Library versions and the hardware used for the experiments are [Information not available in provided materials].

4 Experimental Results and Discussion

This chapter reports the results obtained with the pipeline of Chapter 3. All numbers are taken from, or computed directly from, the result files generated by the code (threshold_metrics.csv, ground_truth_pairs.csv, hash_database.csv and transform_log.csv). The per-transformation statistics in Section 4.4 and the distance statistics in Section 4.3 were computed from these files.

4.1 Experimental Setup

Table 4.1 summarises the experiment. Each of the 50 source images was transformed with 96 parameter settings, giving 4,800 transformed images. Hashes were computed for all 4,850 images (50 originals and 4,800 transformed images), and 240,000 labelled pairs were evaluated over 200 thresholds.

Table 4.1: Summary of the experimental setup.

Item	Value
Source images	50
Transformation families	10
Transformation settings per source image	96
Transformed images	4,800
Hash codes computed	4,850 (50 originals + 4,800 transformed)
Hash length	512 bits
Backbone	VGG16 (ImageNet weights, no fine-tuning)
Similar pairs (label 1)	4,800
Dissimilar pairs (label 0)	235,200
Total pairs evaluated	240,000
Number of thresholds	200, equally spaced in [0, 1]

4.2 Hashing Results

The hash database contains 4,850 hashes of 512 bits each. Across all hashes, 26.4% of the bits are equal to 1. Fewer than half of the bits are set, which suggests that the pooled feature values are unevenly distributed, with a few large components and many smaller ones, so that fewer than half of them exceed the vector mean. The feature values themselves are not stored in the result files, so this explanation was not verified. In addition, 1,039 of the 4,850 entries repeat a hash string that already appears earlier in the database. Part of this repetition

is expected from the circular median settings that use the same rounded radius (Section 3.8); other causes were not analyzed.

4.3 Hamming Distance Analysis

Figure 4.1 shows the distribution of the normalized Hamming distance for similar and dissimilar pairs, and Table 4.2 gives the corresponding statistics. Similar pairs have a mean normalised distance of 0.1168 (median 0.1055), whereas dissimilar pairs have a mean of 0.3481 (median 0.3516). The two distributions are well separated but not disjoint: the largest distance among similar pairs (0.3398) exceeds the smallest distance among dissimilar pairs (0.0098), and the overlap region determines the errors at any threshold.

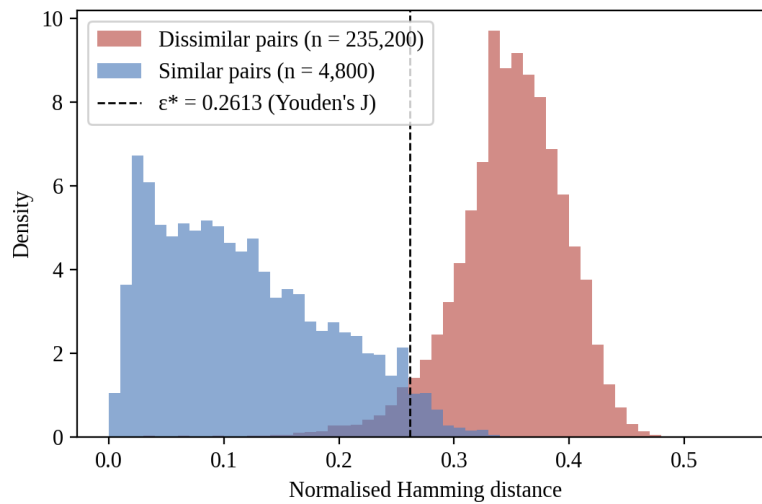


Figure 4.1: Distribution of the normalized Hamming distance for similar and dissimilar pairs. The dashed line marks the Youden-optimal threshold $\varepsilon^* = 0.2613$.

Table 4.2: Normalized Hamming distance statistics for similar and dissimilar pairs.

Pair type	Pairs	Mean	Median	Std. dev.	Min	Max
Similar (label 1)	4,800	0.1168	0.1055	0.0734	0.0000	0.3398
Dissimilar (label 0)	235,200	0.3481	0.3516	0.0502	0.0098	0.5059

4.4 Transformation-Wise Results

Table 4.3 and Figure 4.2 break the similar-pair distances down by transformation. Gamma correction (mean distance 0.037) and colour changes (0.069) alter the hashes least, followed by circular median filtering, brightness, sharpness and rotation, whose mean distances lie between 0.095 and 0.107. Speckle noise (0.176), Gaussian filtering (0.158 and 0.162) and median filtering (0.155) alter the hashes most. The last column of Table 4.3 gives the

fraction of similar pairs that are correctly detected at the threshold ϵ^* selected in Section 4.6: it is 100% for rotation, circular median filtering, gamma correction and colour changes, 98.5% and 99.3% for brightness and sharpness, and between 89.0% and 94.8% for speckle noise, Gaussian filtering and median filtering.

Table 4.3: Similar-pair distance statistics per transformation, and detection rate at $\epsilon^* = 0.2613$.

Transformation	Pairs	Mean	Median	Max	Detected at ϵ^* (%)
Rotation	500	0.1072	0.1035	0.2520	100.0
Speckle noise	500	0.1756	0.1738	0.3301	89.8
Gaussian filter (variance)	500	0.1581	0.1514	0.3262	94.8
Gaussian filter (size)	500	0.1616	0.1641	0.3398	89.0
Median filter	500	0.1553	0.1621	0.3223	92.6
Circular median filter	450	0.0969	0.0947	0.2090	100.0
Gamma correction	500	0.0368	0.0332	0.1074	100.0
Brightness	400	0.0955	0.0654	0.3047	98.5
Colour	500	0.0689	0.0625	0.1914	100.0
Sharpness	450	0.1044	0.0957	0.2734	99.3

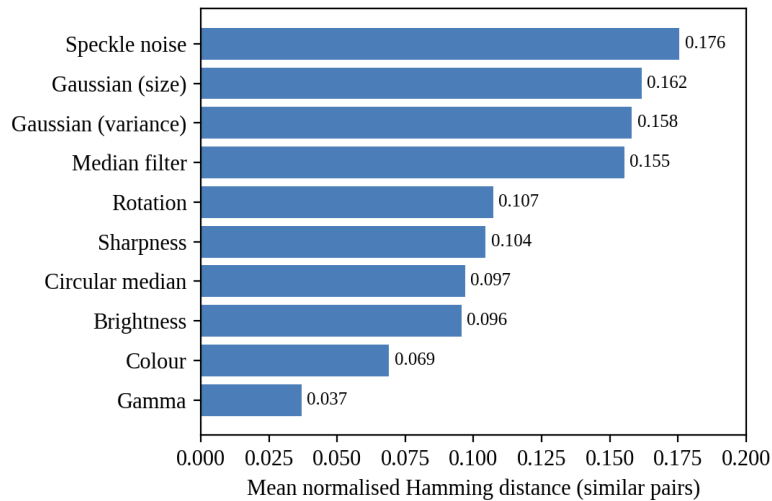


Figure 4.2: Mean normalised Hamming distance of similar pairs for each transformation family.

These results are consistent with the intuition that transformations which change global colour statistics or apply a smooth photometric mapping (gamma, colour) leave the high-level content of the image, and thus the pooled deep features, largely unchanged, whereas transformations that remove or add fine texture (blurring, median filtering, speckle noise)

alter the low-level responses that feed the deeper layers. This explanation is a plausible interpretation and has not been tested separately in this work. The rotation angles used here are small (at most 15°); the result therefore does not say anything about larger rotations.

4.5 Threshold Analysis

Table 4.4 lists the confusion-matrix counts and derived measures at three thresholds of interest, and Table 4.5 lists them for a regular grid of thresholds (the nearest of the 200 swept values is shown). Figure 4.3 plots TPR, FPR, precision and F1 as functions of ϵ . As ϵ grows, TPR rises quickly, reaching 0.9629 at $\epsilon = 0.2613$, where FPR is 0.0470. Precision stays close to 0.78 for ϵ between about 0.03 and 0.13 and then falls steadily. Because dissimilar pairs outnumber similar pairs 49 to 1, even a small FPR produces many false positives in absolute terms.

Table 4.4: Performance at selected operating points.

Criterion	ϵ	TP	FP	FN	TN	TPR	FPR	Prec.	F1
Max. F1	0.1658	3542	1355	1258	233845	0.7379	0.0058	0.7233	0.7305
EER	0.2563	4592	10059	208	225141	0.9567	0.0428	0.3134	0.4722
Youden's J	0.2613	4622	11060	178	224140	0.9629	0.0470	0.2947	0.4513

Table 4.5: Performance over a regular grid of thresholds (nearest swept value).

ϵ	TP	FP	FN	TN	TPR	FPR	Precision	F1
0.0503	1084	298	3716	234902	0.2258	0.0013	0.7844	0.3507
0.1005	2286	647	2514	234553	0.4763	0.0028	0.7794	0.5912
0.1508	3329	1144	1471	234056	0.6935	0.0049	0.7442	0.7180
0.2010	4021	2846	779	232354	0.8377	0.0121	0.5856	0.6893
0.2513	4544	8692	256	226508	0.9467	0.0370	0.3433	0.5039
0.3015	4773	33990	27	201210	0.9944	0.1445	0.1231	0.2191

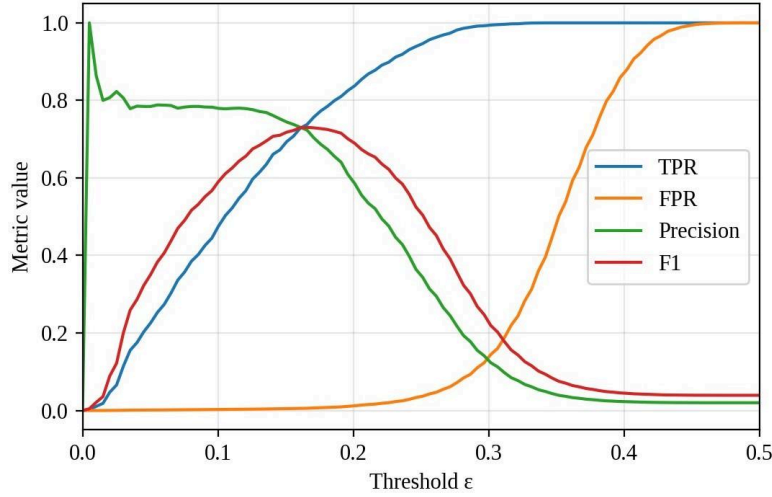


Figure 4.3: TPR, FPR, precision and F1 as functions of the decision threshold ϵ .

The F1 score reaches its maximum of 0.7305 at $\epsilon = 0.1658$ (TPR 0.7379, FPR 0.0058, precision 0.7233). The Youden-optimal threshold $\epsilon^* = 0.2613$ instead maximises the difference $\text{TPR} - \text{FPR}$, favouring recall; at this point the F1 score is only 0.4513 because precision falls to 0.2947. Which operating point is appropriate depends on the application: a copy-detection system that must avoid false alarms would prefer a smaller threshold, whereas a system that must not miss modified copies would accept the lower precision.

4.6 ROC Curve, AUC and EER

Figure 4.4 shows the ROC curve obtained by the threshold sweep. The curve rises steeply towards the top-left corner and lies far above the diagonal that represents random guessing. The area under the curve is 0.991. The EER, that is, the false positive rate at the threshold where FPR and FNR are closest ($\epsilon = 0.2563$; FPR = 0.0428, FNR = 0.0433), is approximately 4.3%. The Youden-optimal threshold is $\epsilon^* = 0.2613$ with TPR = 0.9629 and FPR = 0.0470. Table 4.6 collects these summary measures. Because the AUC and EER are computed from 200 discrete thresholds, they are approximations of the continuous quantities.

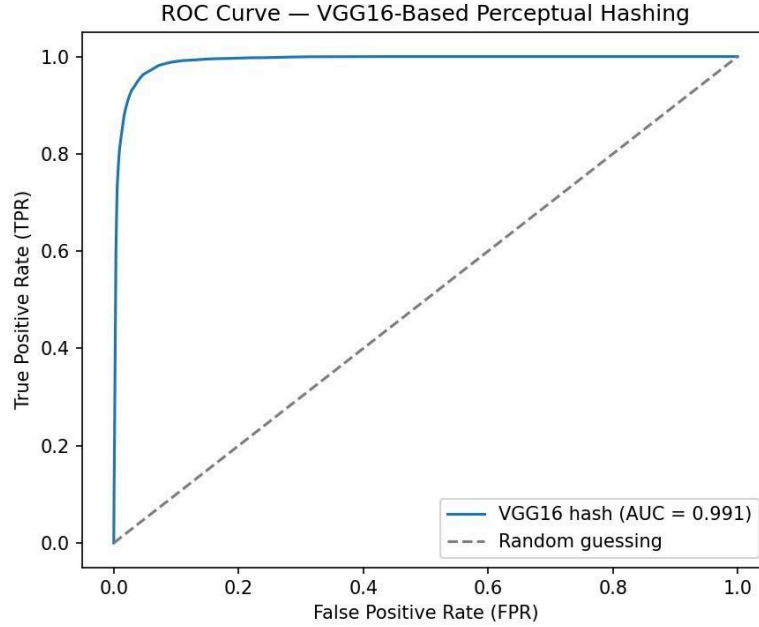


Figure 4.4: ROC curve of the VGG16-based perceptual hashing scheme (AUC = 0.991).

Table 4.6: Summary of the evaluation measures.

Measure	Value
AUC (trapezoidal rule)	0.991
EER (FPR at the point where $\text{FPR} \approx \text{FNR}$)	0.0428 (approximately 4.3%), at $\varepsilon = 0.2563$
Youden-optimal threshold ε^*	0.2613 (TPR = 0.9629, FPR = 0.0470)
Maximum F1	0.7305, at $\varepsilon = 0.1658$

4.7 Confusion Matrix

Figure 4.5 and Table 4.7 show the confusion matrix at $\varepsilon^* = 0.2613$. Of the 4,800 similar pairs, 4,622 are correctly identified and 178 are missed (TPR = 0.9629). Of the 235,200 dissimilar pairs, 224,140 are correctly rejected and 11,060 are wrongly accepted (FPR = 0.0470). Consequently the precision at this threshold is $4,622 / (4,622 + 11,060) = 0.2947$.

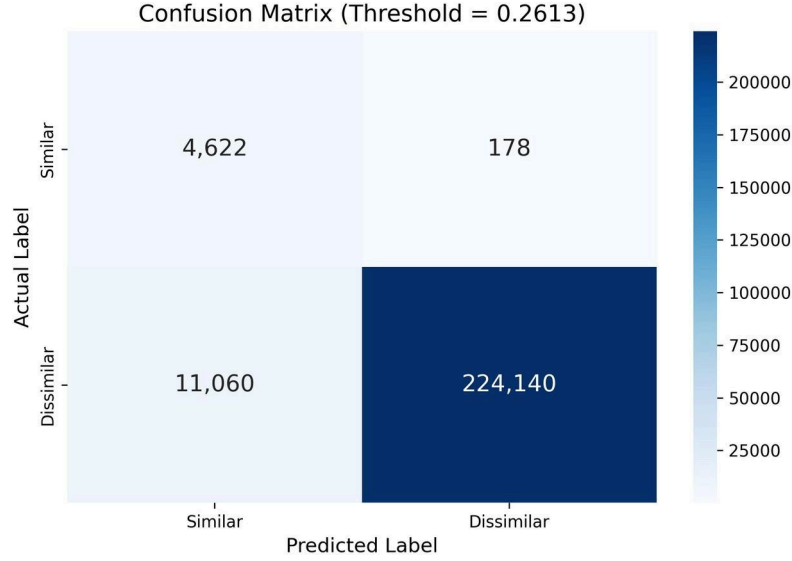


Figure 4.5: Confusion matrix at the Youden-optimal threshold $\epsilon^* = 0.2613$ (rows: actual class; columns: predicted class).

Table 4.7: Confusion matrix at $\epsilon^* = 0.2613$.

	Predicted similar	Predicted dissimilar
Actual similar	TP = 4,622	FN = 178
Actual dissimilar	FP = 11,060	TN = 224,140

4.8 Discussion

The results show that the hash derived from a frozen VGG16 separates transformed copies from different images well in terms of the ranking of distances (AUC 0.991, EER about 4.3%), even though no training for the hashing task was performed. Robustness varies by transformation: the scheme is very stable under gamma correction, colour changes and small rotations, and less stable under speckle noise, blurring and median filtering, which suggests that texture-altering operations are the main weakness of the current hash. The results also expose the cost of the class imbalance inherent in pair-based evaluation. With 49 dissimilar pairs for every similar pair, an FPR of 4.7% translates into 11,060 false positives, and precision at the Youden-optimal threshold is below 30%. Practical use would therefore require either a smaller threshold, at the cost of missing more transformed copies (at $\epsilon = 0.1658$, TPR = 0.7379), or a second verification stage.

Several limitations should be noted. (i) The dataset has only 50 source images, and their diversity is not characterised beyond the examples in Figure 3.2. (ii) Every transformed variant is labelled similar regardless of the severity of the transformation, so strong settings

(for example a large median window) count as similar even though they may alter the image considerably. (iii) Dissimilar pairs combine a source image with *transformed* images of other sources; pairs of two unmodified different images are not evaluated separately. (iv) Results come from a single run of a deterministic pipeline, and no confidence intervals are reported. (v) No other attacks (such as cropping or scaling) and no other methods are evaluated, so the results should not be interpreted as a comparison with the methods discussed in Chapter 2.

5 Conclusion and Future Work

5.1 Conclusion

This thesis presented a perceptual image hashing scheme that maps an image to a 512-bit hash by passing it through the convolutional base of an ImageNet-pretrained VGG16, applying global average pooling, and binarising the 512-dimensional feature vector around its mean. Image similarity is judged by the normalised Hamming distance between hashes. The scheme was evaluated on 50 images transformed with ten families of content-preserving operations (4,800 similar and 235,200 dissimilar pairs). The threshold sweep yielded an AUC of 0.991 and an EER of about 4.3%; at the Youden-optimal threshold of 0.2613 the TPR was 96.3% and the FPR 4.7%. The scheme was most robust to gamma correction, colour changes and small rotations and least robust to speckle noise, Gaussian filtering and median filtering. Given the low precision at high recall, the scheme is best regarded as a baseline that demonstrates the usefulness of pretrained CNN features for perceptual hashing rather than as a finished system.

With respect to the objectives of Section 1.5, the literature survey, the design and implementation of the pipeline, the construction of the evaluation dataset and the threshold-based evaluation have been completed. The use of the hash in a downstream application has not been tested.

5.2 Future Work

The following directions follow from the limitations identified above:

- **End-to-end training.** Fine-tune the VGG16 backbone with a contrastive or triplet loss together with a quantisation penalty, instead of relying on features transferred from ImageNet classification. The current code uses a frozen pretrained backbone and does not implement this.
- **Backbone comparison.** Evaluate VGG19, which is already supported by the feature extractor, and other architectures under the same protocol.
- **Broader evaluation.** Use a larger and more diverse image set, add further attacks (for example cropping and scaling) and combinations of attacks, and report confidence intervals.
- **Threshold and binarisation study.** Investigate alternative binarisation rules and operating points that improve precision at a given recall.
- **Applications.** Test the hash in copy detection and image authentication, which are the applications motivating this work.

References

- [1] L. Du, A. T. S. Ho and R. Cong, “Perceptual hashing for image authentication: A survey,” *Signal Processing: Image Communication*, vol. 81, Feb. 2020.
- [2] L. Yuan, T. Wang, X. Zhang, F. E. H. Tay, Z. Jie, W. Liu and J. Feng, “Central similarity quantization for efficient image and video retrieval,” Aug. 2019.
- [3] C. Qin, E. Liu, G. Feng and X. Zhang, “Perceptual image hashing for content authentication based on convolutional neural network with multiple constraints,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, pp. 4523–4537, Nov. 2021.
- [4] M. Srivastava, J. Siddiqui and M. A. Ali, “Robust image hashing based on statistical features for copy detection,” in *Proc. IEEE Uttar Pradesh Section International Conference on Electrical, Computer and Electronics Engineering*, 2017, pp. 490–495.
- [5] Z. Huang and S. Liu, “Robustness and discrimination oriented hashing combining texture and invariant vector distance,” in *Proc. 26th ACM International Conference on Multimedia*, 2018, pp. 1389–1397.
- [6] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” arXiv:1409.1556, 2014.
- [7] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.